

Software Systems

Session 1

Y. Raghu Reddy & Karthik Vaidhyanathan

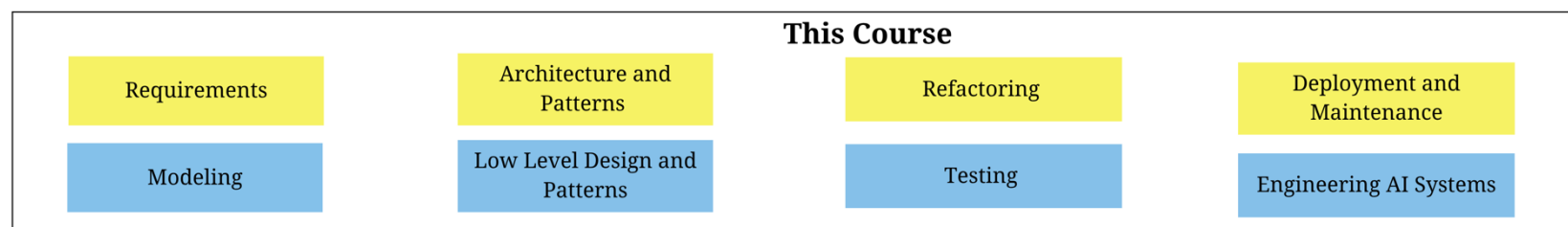
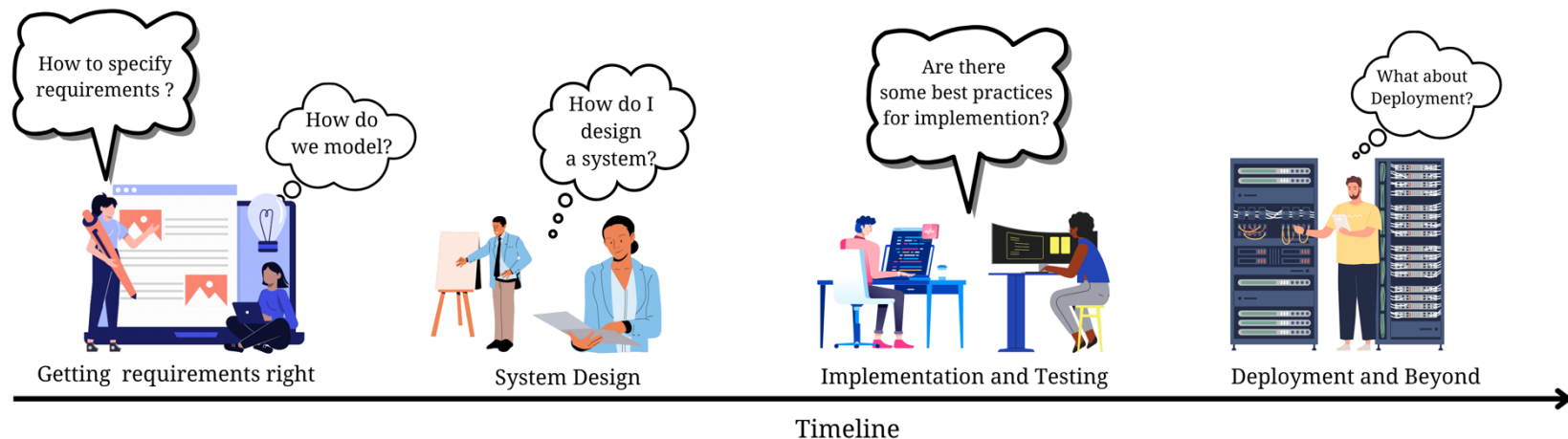
Software Engineering Research Center
IIIT Hyderabad, India



Software Systems - What can we expect?

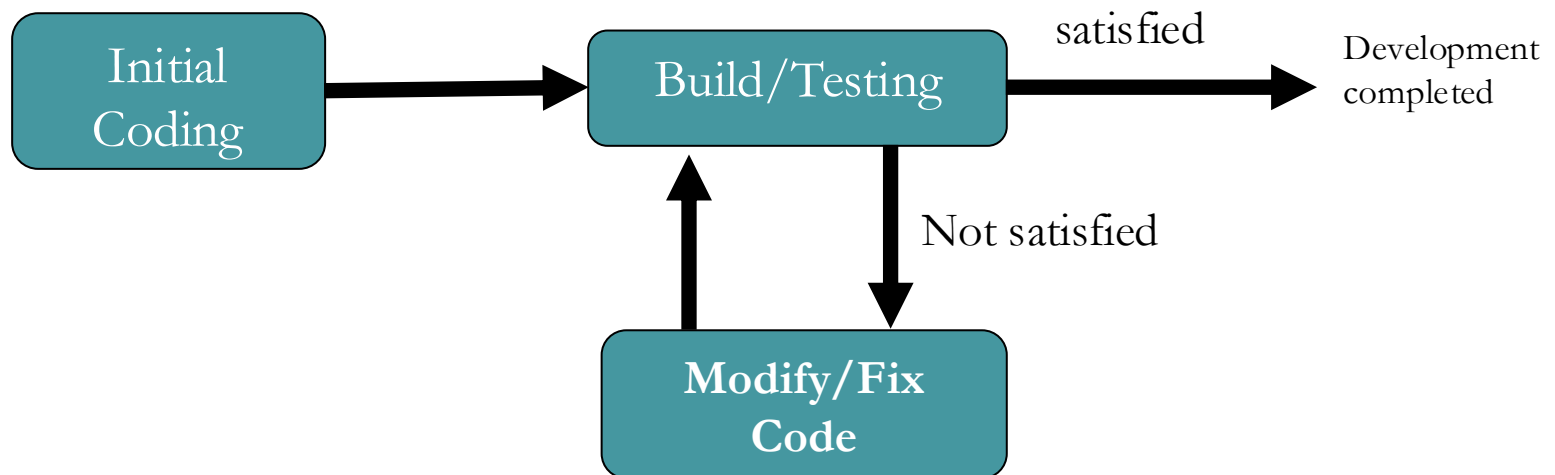
By the end of this course you

- should be able to understand the lifecycle of software and various process models
- should be able to model and build software using a specific development lifecycle
- should be able to assess the qualities of existing software and improve as needed
- should be able to understand the use of AI in software development



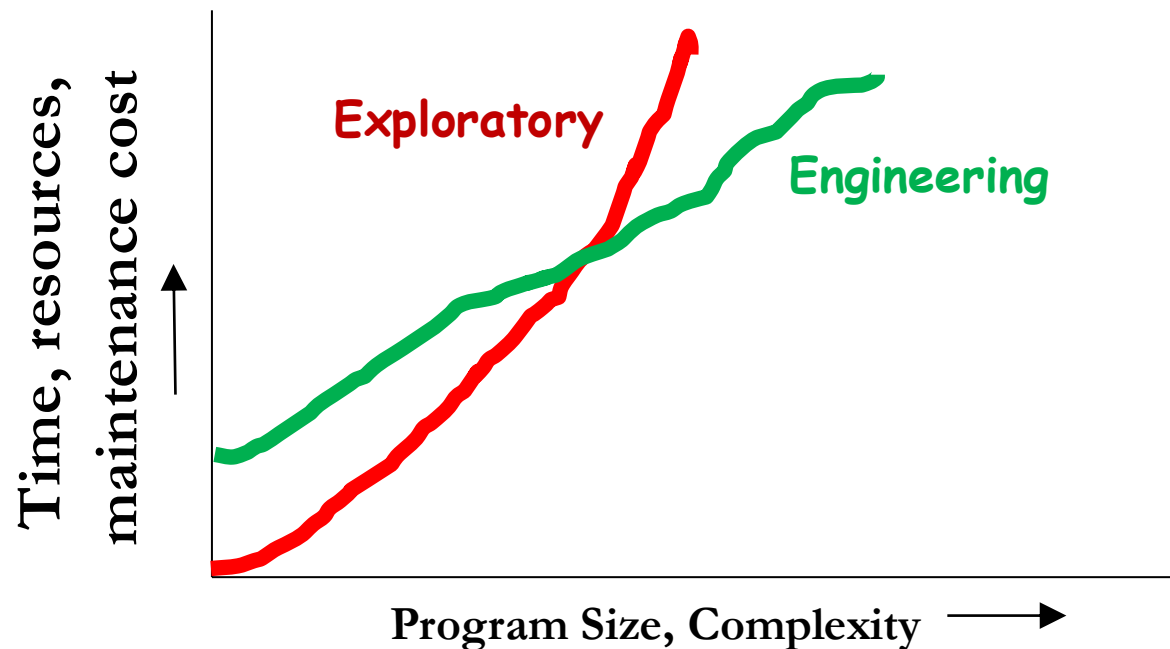
Exploratory programming

- The early programmers used an exploratory (also called build and fix) style.
 - In the build and fix (exploratory) style, initial program is quickly developed.
 - The different imperfections that are subsequently noticed are fixed.



What is Wrong with the Exploratory Style?

- Can successfully be used for very small programs only
- Large programs can become unmaintainable
- Time and Effort required to develop a product can grow exponentially with program size



PROGRAMS	SOFTWARE PRODUCTS
Usually small in size	Large
Author himself is sole user	Large number of users
Single developer	Team of developers
Lacks proper user interface	Well-designed interface
Lacks proper documentation	Well documented & user-manual prepared
Ad hoc development/Exploratory	Systematic development

Why Engineer Software Systems?

- The ad hoc approach/opportunistic approach breaks down when size of software increases.
- Building large complex (software) systems require a systematic approach
 - Exponential growth in complexity and difficulty level with size.
 - Large systems cannot be understood by one person
 - Requires team work
 - Achieve sufficient quality (e.g. Maintainability, Usability, etc)
- Complex software requires adoption of engineering principles
 - Decomposition
 - Abstraction
 - ...
- Also learn methods/tools/techniques of
 - Specification, design, user interface development, testing, project management, etc.

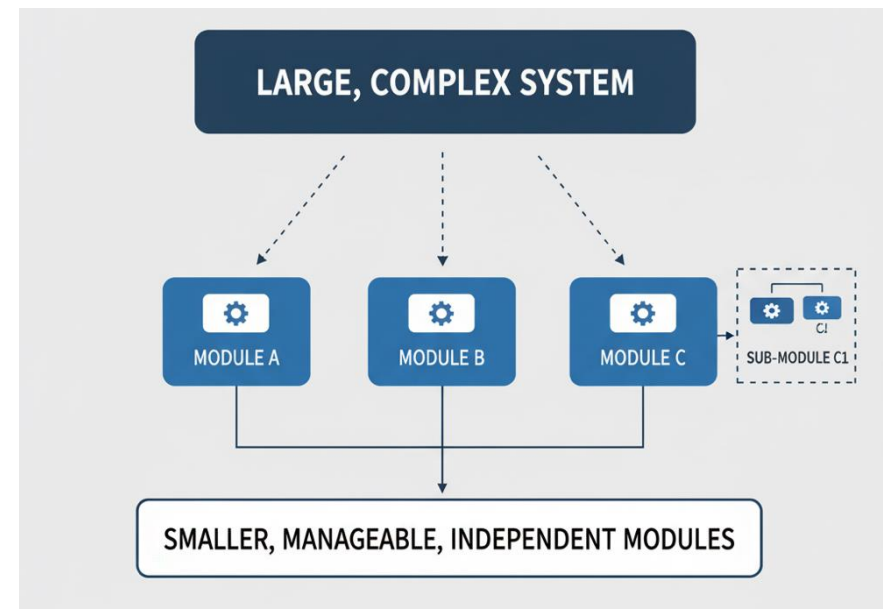
Fundamental principles in Engineering Software Systems

Decomposition breaks a complex system into smaller, manageable parts (modules, components, functions) that can be developed, understood, and can possibly be maintained independently

Purpose: Reduce complexity, enable parallel work, improve testability and reuse.

Types:

- Functional decomposition: split by features or behaviors (e.g., auth, billing).
- Layered decomposition: separate concerns by layer (UI, business logic, data).
- Modular/component decomposition: self-contained modules or services.



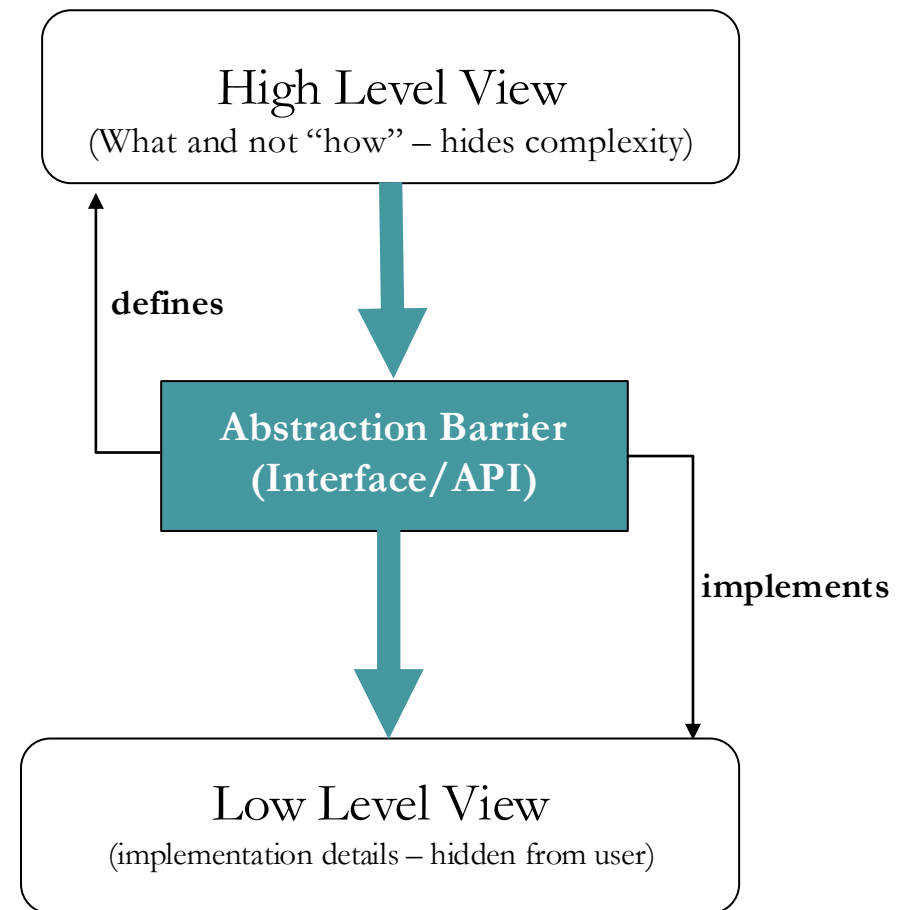
Fundamental principles in Engineering Software Systems

Abstraction hides irrelevant details and exposes only the essential features needed at a particular level of reasoning.

Purpose: Manage complexity, improve readability, enable reuse, and reduce coupling.

Examples:

- Functions/methods: expose behavior through an interface, hide implementation.
- Classes/objects: public API vs. private fields/methods.
- Modules/APIs: services with well-defined contracts.
- Abstract data types: stacks, queues (what they do, not how).



Thank you !

What do these have in common?

- They all need a *lot* of software to operate.
- The Boeing 777 flies with over 5,000,000 lines of code on-board.
- A typical top-level game probably has more than 1 Million SLOC (source lines of code)
- These are large software systems that cannot be thought of one class at a time. The software engineer needs to think about the **design** at different levels – from a small module up to the entire system.



Design

$1 \text{ SLOC/min/SE} * 60 \text{ min/hr} * 40 \text{ hrs/wk} =$
 $2,400 \text{ SLOC/wk/SE}$

$2,400 \text{ SLOC/wk/SE} * 50 \text{ wk/year} =$
 $120,000 \text{ SLOC/year/SE}$

$2 * 10^6 \text{ SLOC} / 1.2 * 10^5 \text{ SLOC/yr/SE} =$
 $\sim 17 \text{ SEs for the year}$

Teamwork



The software engineer's daily job is to answer questions about the software system...

- How can I help the customer? What is required to solve the customer's problem?
- How will the user interact with the system?
- What operating system, language, hardware is going to be used?
- What is the overall software system structure and how do different components interact with each other?
- What code do I have to write?
- How do I organize my team so we are effective?
- Can we finish the game in time to have it on the shelves for Christmas shopping?

To answer those questions, the software engineer must work with many people.

- Customers asking for the system
- People who will use the system
- Domain experts: banking, avionics, security, medical, scientists, ...
- Engineers from other engineering disciplines
- Most closely with the other software engineers on the project

Communication

Yes, software engineers get their hands dirty writing programs using the latest technologies and techniques.



SOFTWARE DEPLOYMENT FAILURE

KNIGHT CAPITAL GROUP - AUGUST 1, 2012

ROOT CAUSE:

Faulty software deployment.
Configuration error in trading application.
Poor release management.



IMPACT:

US\$440 MILLION LOSS
in 45 minutes.
Company required
emergency financing & sale.



Process

Electronic Trading Firm

Complex!!!

One small upgrade. Easy.

One 45 minute outage.

Estimated millions in lost revenue.

Ended up in selling the company

A critical exam

This is a financial-critical system.

The team needed a better understanding of the configuration and release process for developing a financial-critical system.

Software Development Lifecycle (SDLC) – connecting design, people and process

- SDLC formalizes programming, design, teamwork, communication, and process across phases
- A software project will never succeed if activities are not coordinated:
 - one engineer starts writing code,
 - another concentrates on writing the test document first,
 - yet another engineer first defines the file structure
 - another defines the I/O for his portion first
- Adherence can lead to accurate status reports

SDLC – Phases and how do the keywords fit !



Feasibility study (involves business case)



Requirements Engineering: relies on **Teamwork** and **Communication** to gather correct needs and define scope.



Design: directly uses **design** principles to create architecture and component plans that guide Programming.



Implementation: **Programming** converts design into working software; collaboration and code standards keep quality high.

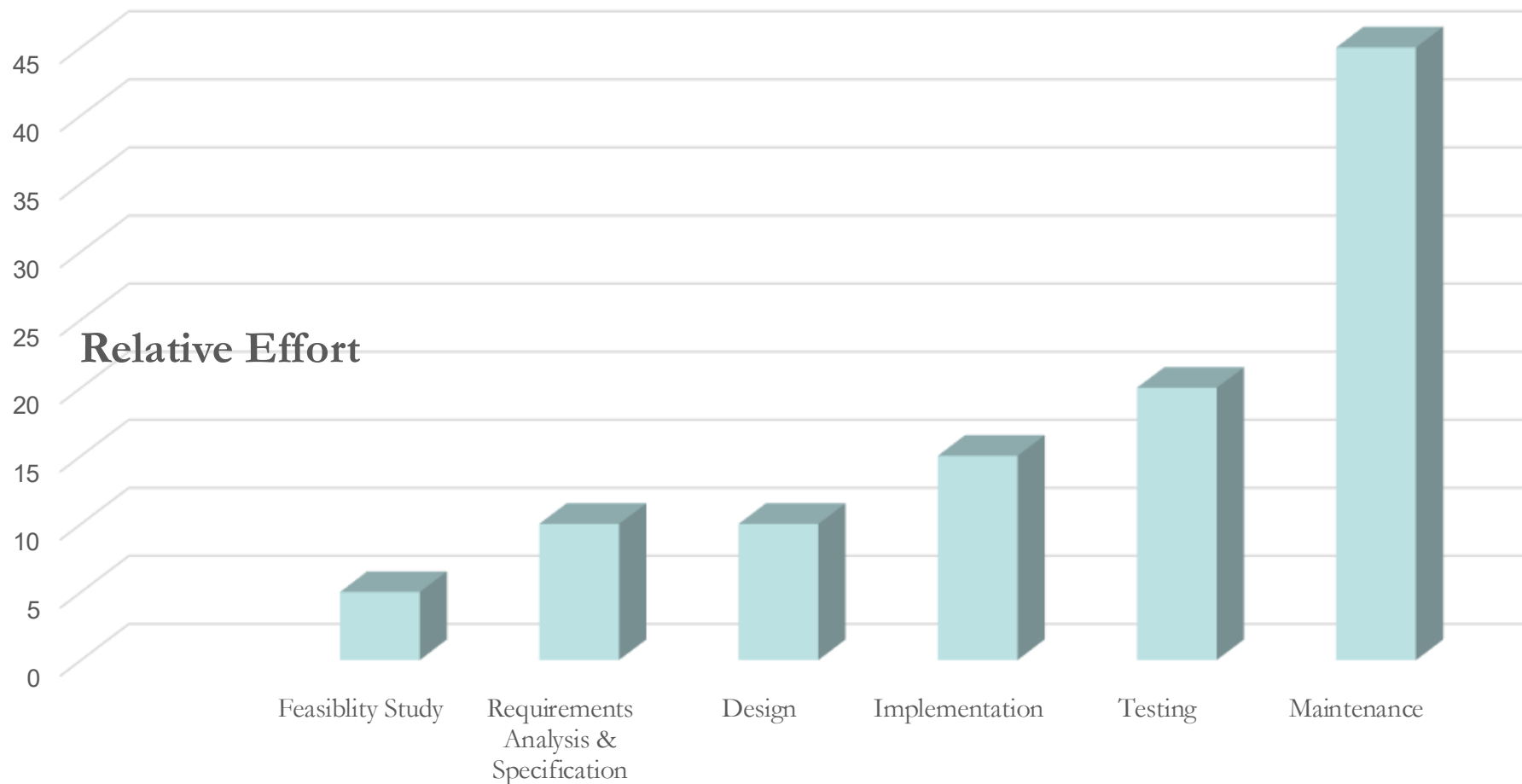


Integration & Testing: Process (Incremental Integration, automated tests, deployment) and **teamwork** ensure reliable integration and bug detection.



Deployment & Maintenance: **Process** governs releases; **Communication** and **Teamwork** manage incidents and iterative improvements.

SDLC Phases – Relative Effort



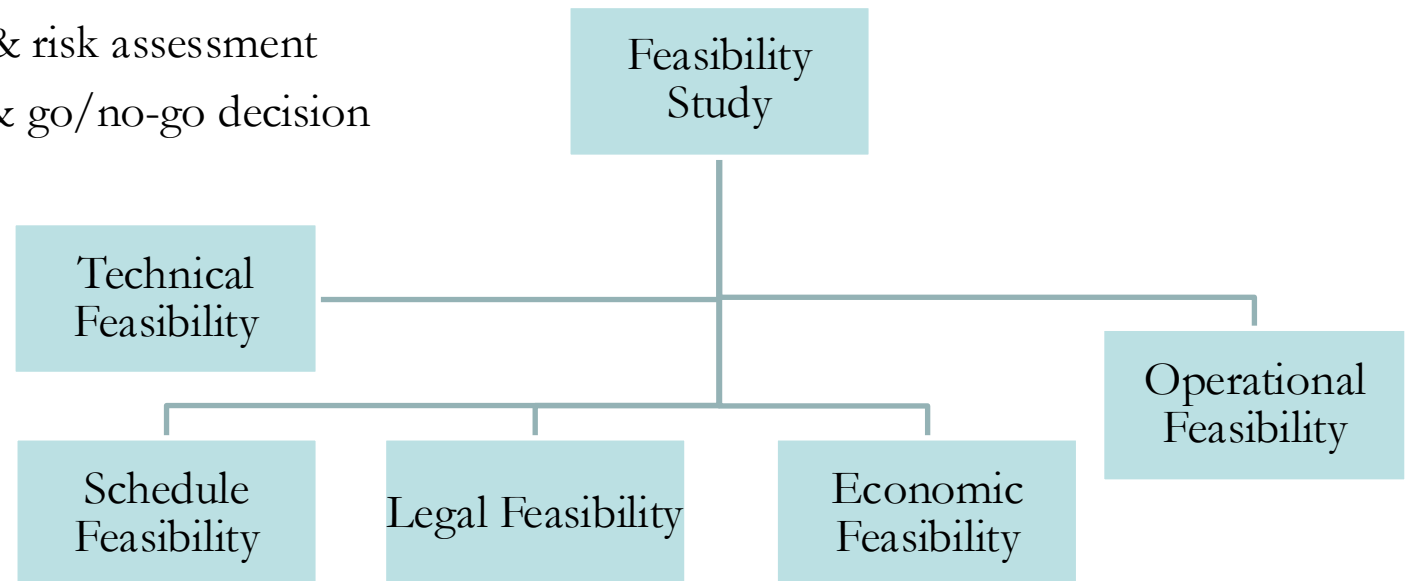
Thank you !

SDLC - Feasibility Study

Aim of feasibility study: Determine whether the proposed system is viable and worth pursuing

Key Activities:

- Stakeholder interviews & requirements scoping
- Technical feasibility
- Economic (cost–benefit) analysis
- Schedule feasibility & preliminary estimation
- Operational feasibility
- Legal/compliance & risk assessment
- Recommendation & go/no-go decision



Requirements Engineering

Elicit, analyze, specify, validate, and manage stakeholder needs so the system built meets intended purpose and constraints

Consists of distinct activities:

- **requirements gathering and analysis**
- **requirements specification**
- verification & validation
- requirements management & change control

Design

Transform validated requirements into a robust, maintainable, and implementable architecture and detailed designs that guide development and testing

High-level design:

- decompose the system into [modules/components](#)
- represent invocation relationships among the modules/components
- Specify constraints

Detailed design (Low-level design):

- data structures, interfaces and algorithms for each module are designed;
- API contracts, sequence/collaboration diagrams, and database schemas

Implementation – Building the system

Convert design modules into codified components/classes following coding standards/conventions

Coding Standards:

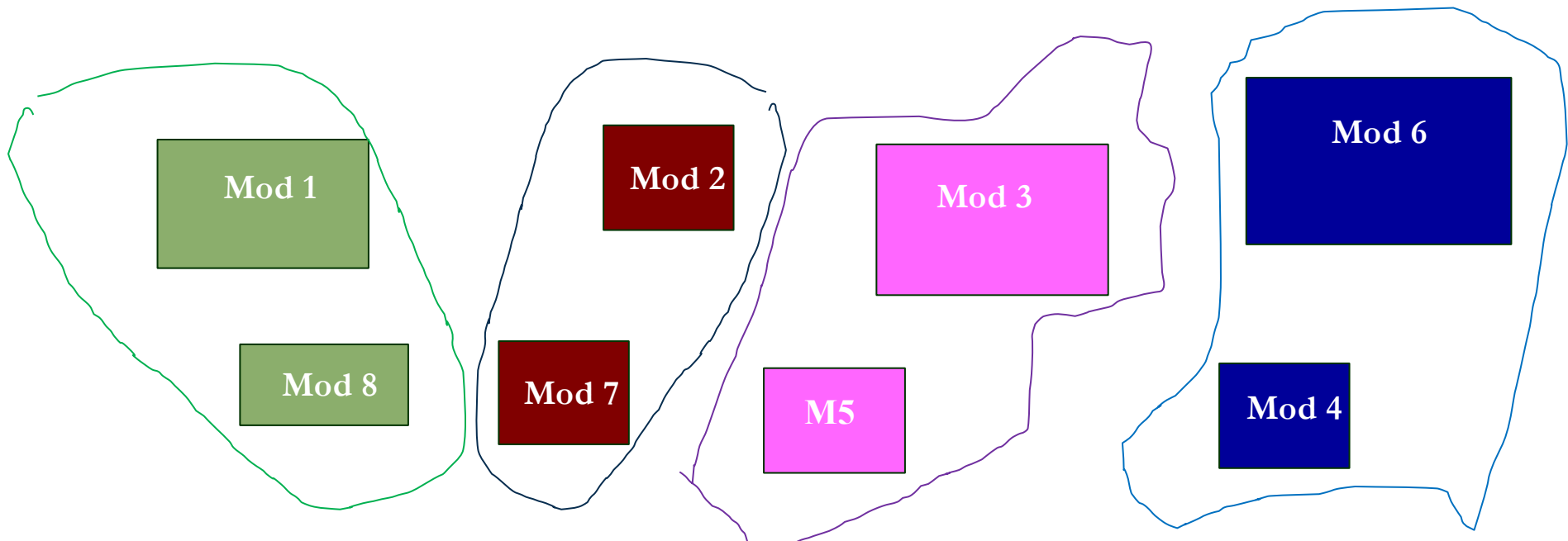
- Enforce team-specific coding standards (naming, layout, function size) for all programmers.
 - Consistency boosts readability, maintainability, and good practices across the codebase.
-
- Microsoft .NET/C# Guidelines — <https://learn.microsoft.com/dotnet/csharp/fundamentals/coding-style/>
 - Python PEP 8 — <https://peps.python.org/pep-0008/>
 - Oracle Java Code Conventions / Google Java Style — <https://google.github.io/styleguide/javaguide.html>
 - CERT Secure Coding (C/C++/Java) — <https://wiki.sei.cmu.edu/confluence/display/seccode/SEI+CERT+Coding+Standards>

The end product of implementation phase:

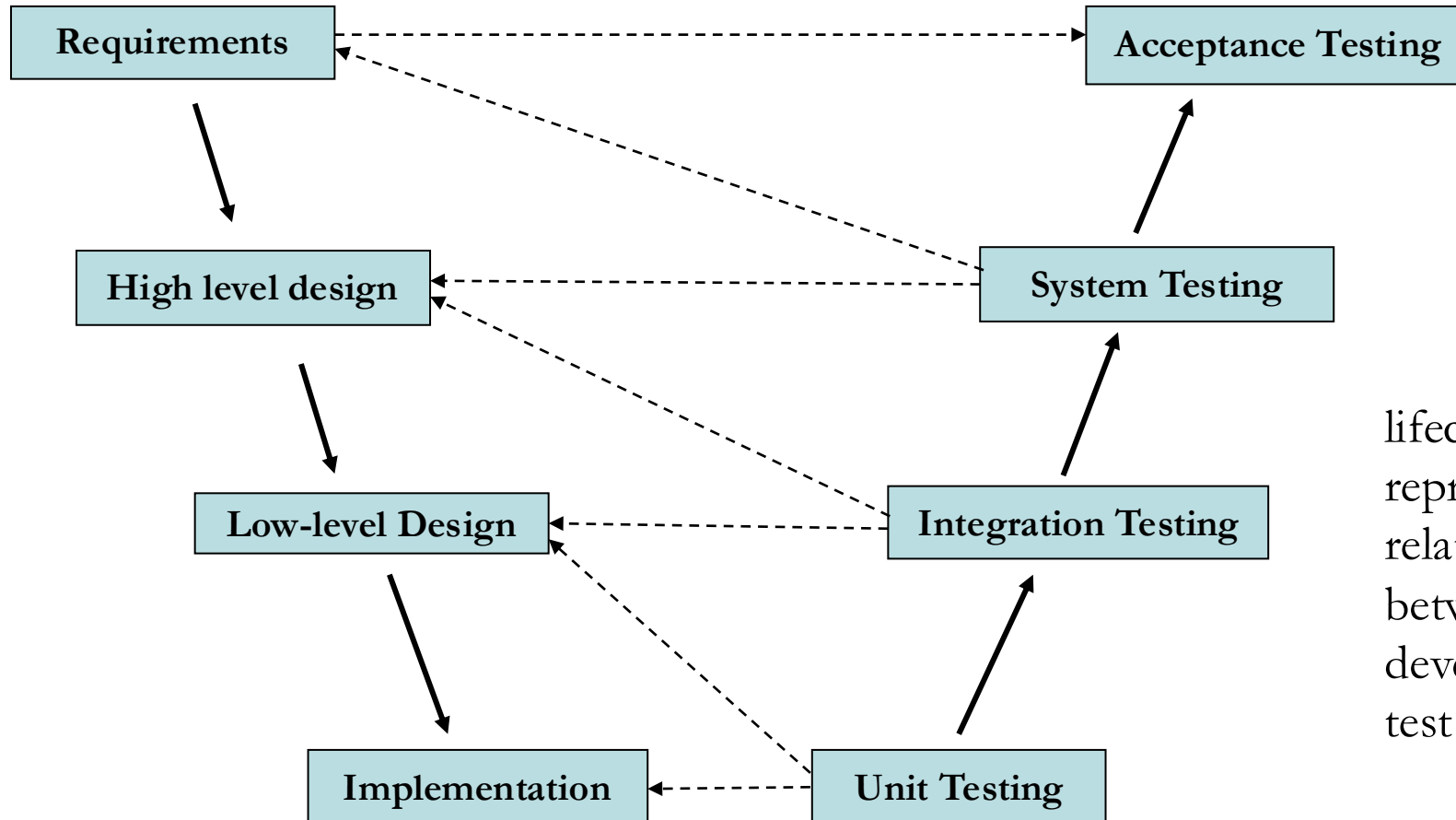
- transform design into code
- a set of program modules that can be tested individually (**unit testing**)

Integration and Testing

- Different modules are integrated in a planned manner:
 - modules are almost never integrated in one shot.
 - Normally integration is carried out through a number of steps.
- During each integration step,
 - the integrated system is tested.



V- Model



lifecycle view
representing
relationships
between
development and
test phases

Maintenance

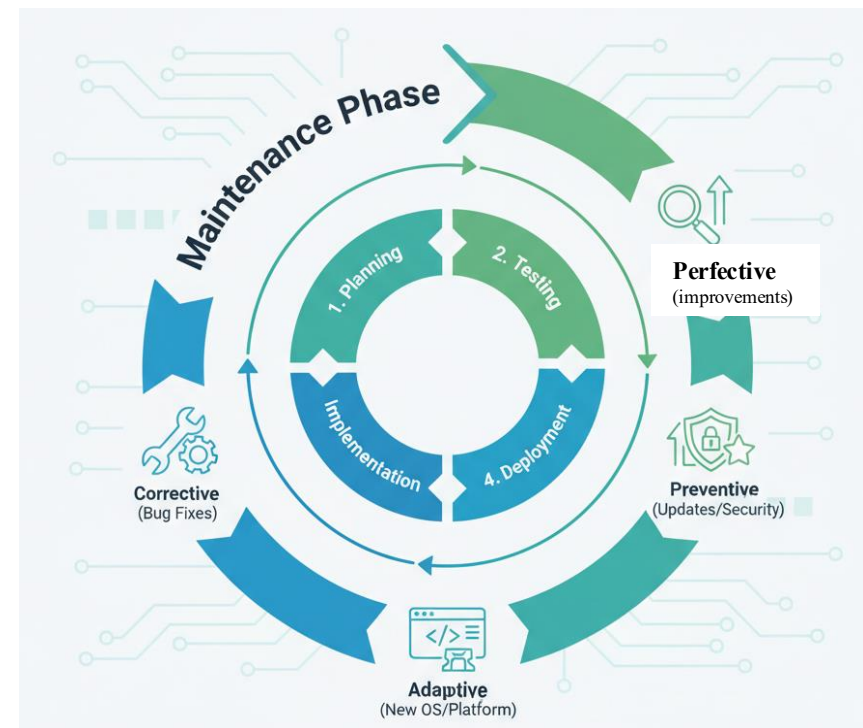
- Maintenance of any software product:
 - requires much more effort than the effort to develop the product itself

Preventive maintenance: Defect prevention

Corrective maintenance: Bug fixes

Perfective maintenance: Improvements/enhancements

Adaptive maintenance: Port software to a new environment

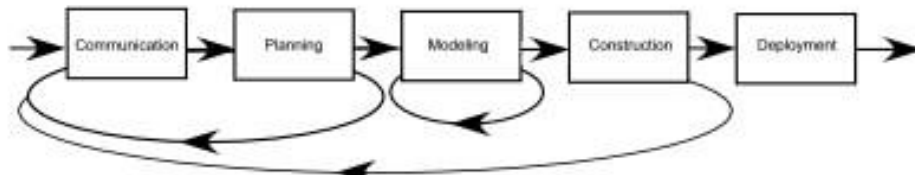


Thank you !

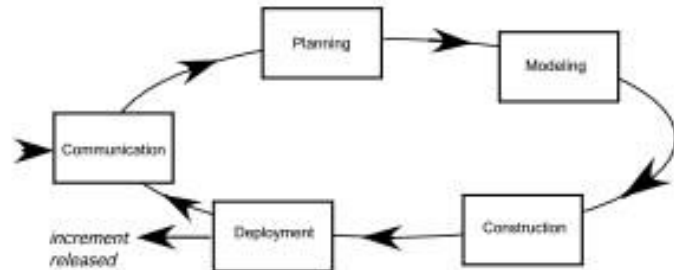
Software Lifecycle - Process Flow



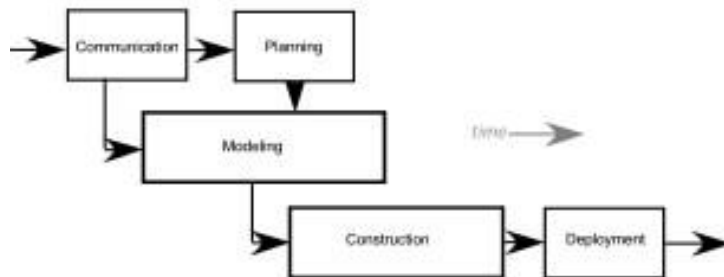
(a) linear process flow



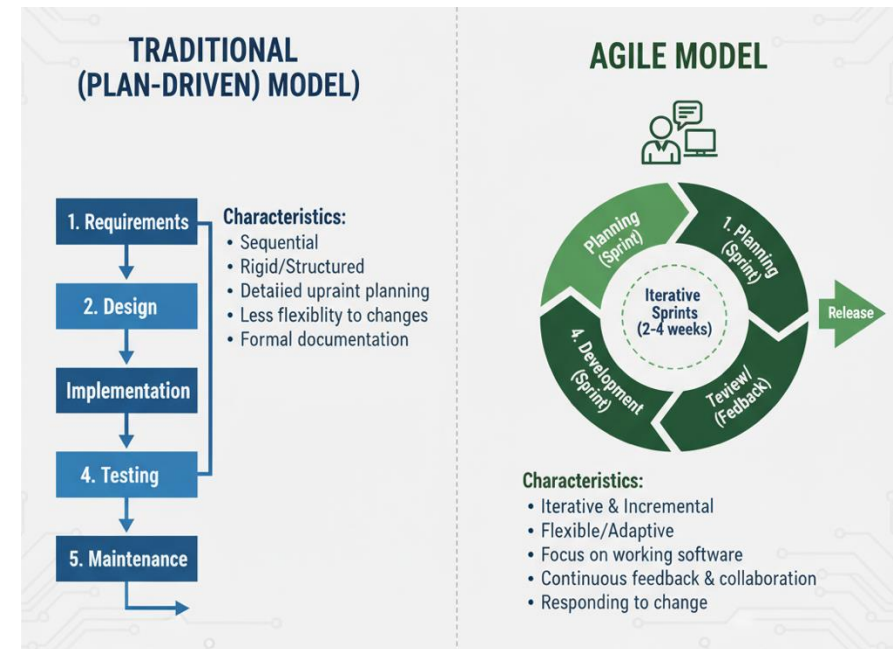
(b) iterative process flow



(c) evolutionary process flow



(d) parallel process flow

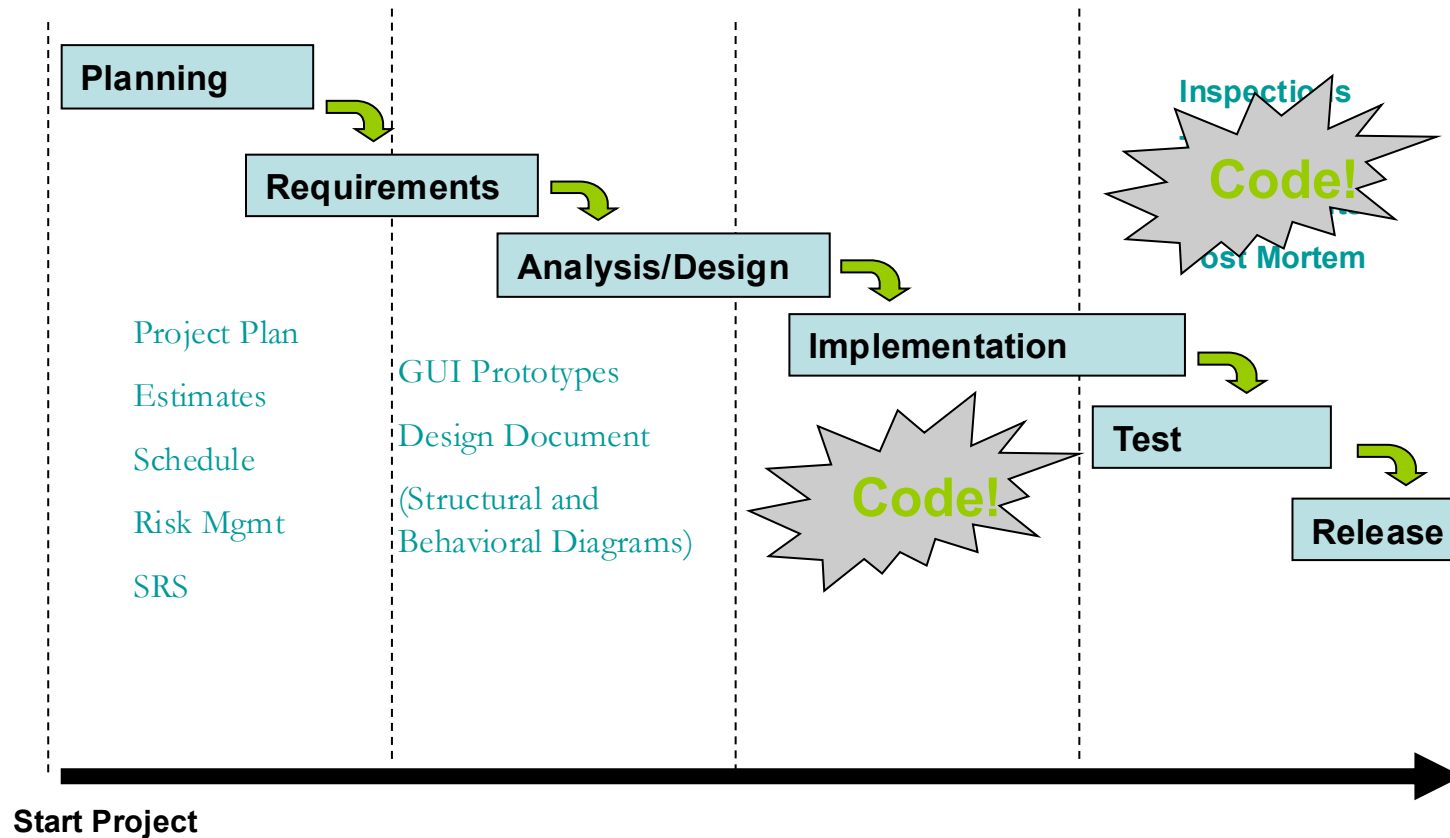


Process Models

Many life cycle models have been proposed

- ▶ Traditional Models (plan-driven)
 - ▶ Classical waterfall model
 - ▶ Iterative waterfall
 - ▶ Evolutionary
 - ▶ Prototyping
 - ▶ Spiral model
 - ▶ Rational Unified Process (RUP)
- ▶ Agile Models
 - ▶ eXtreme Programming (XP)
 - ▶ Scrum
 - ▶ Crystal
 - ▶ Feature-Driven Development (FDD)
 - ▶ ...

Traditional SDLC (e.g. waterfall process)

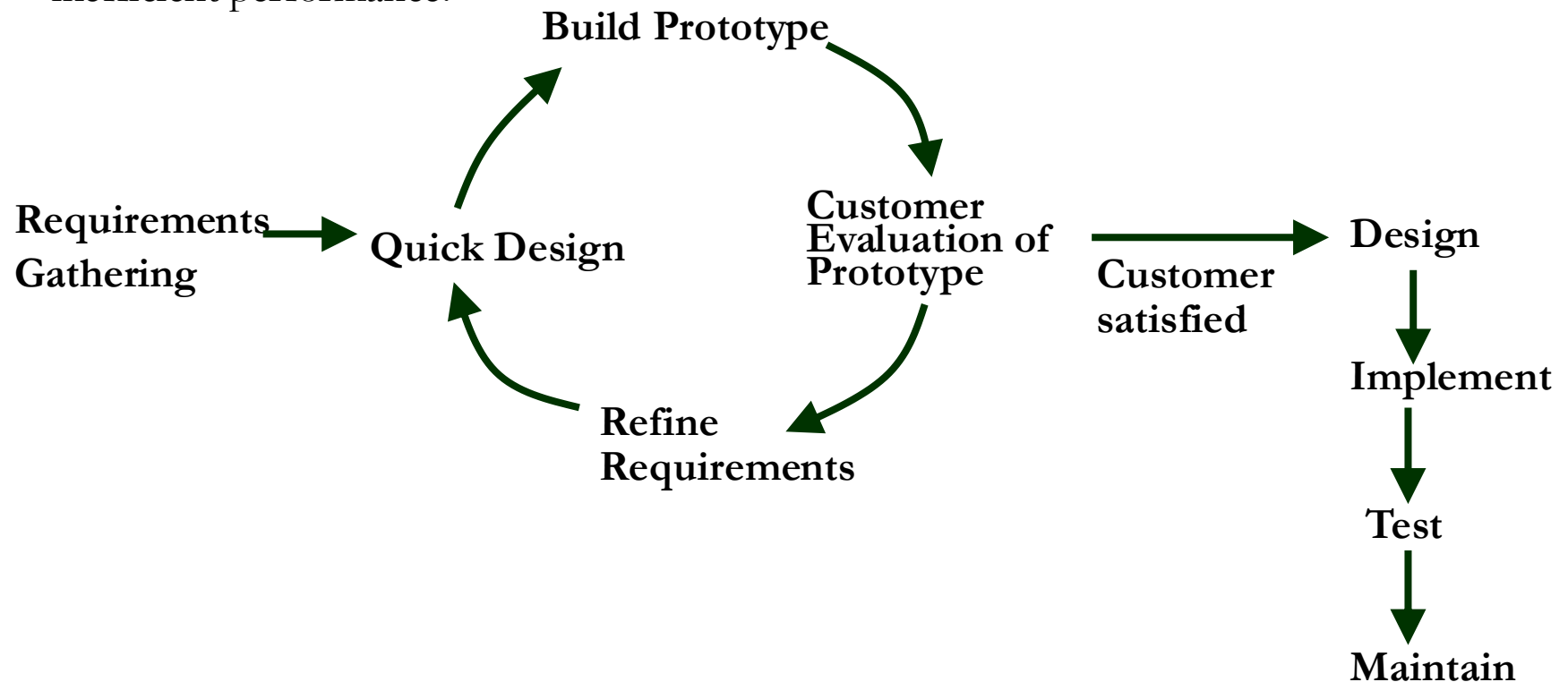


Challenges with waterfall model

- Lightweight applications/heavyweight process
- Document intensive (perceived)
- Less flexible design
- Big bang approach to coding/integration
- Testing short-shifted
- One-shot delivery opportunity
- Limited opportunity for process improvement

Prototyping process model

- Before starting actual development,
 - a working prototype of the system should first be built.
- A prototype is a toy implementation of a system:
 - limited functional capabilities,
 - low reliability,
 - inefficient performance.



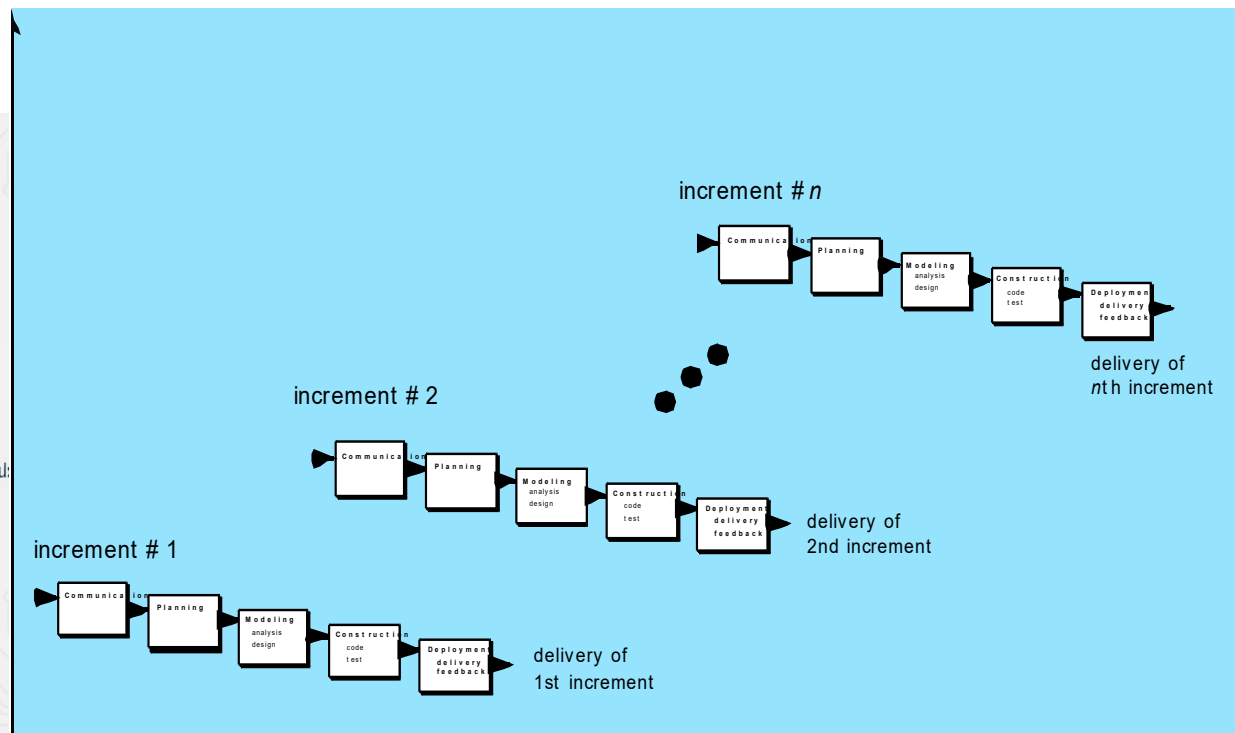
Evolutionary process model

- Evolutionary model (aka successive versions or incremental model):
 - The system is broken down into several modules which can be incrementally implemented and delivered.
- First develop the core modules of the system.
- The initial product skeleton is refined into increasing levels of capability:
 - by adding new functionalities in successive versions.

EVOLUTIONARY MODEL WITH ITERATION

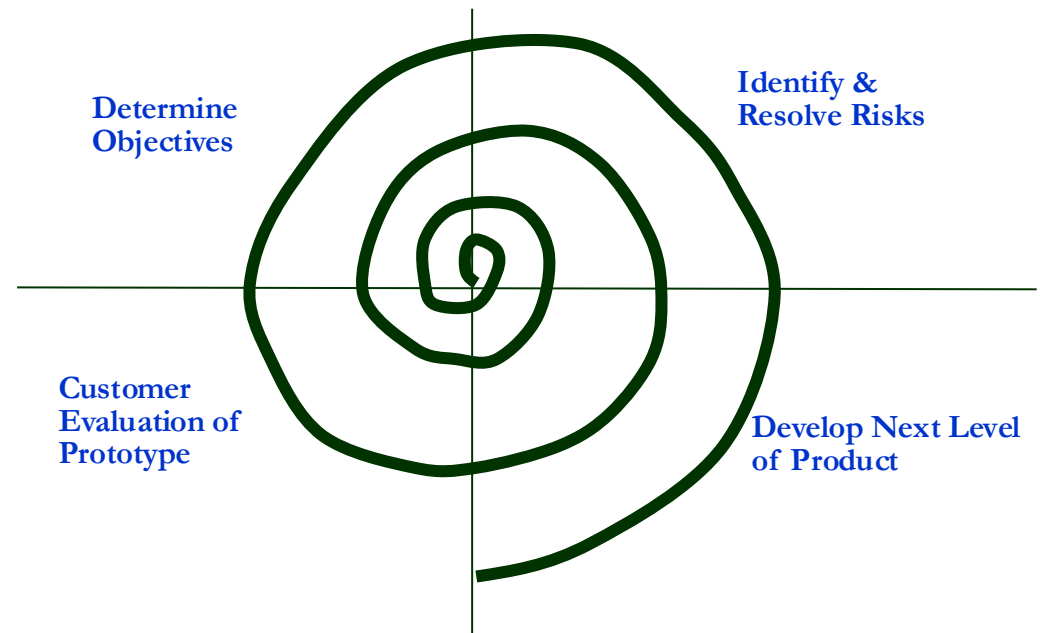
(Iterative Incremental Model)

- Many organizations use a combination of iterative and incremental development:
 - a new release may include new functionality
 - existing functionality from the current release may also have been modified.



SPIRAL MODEL

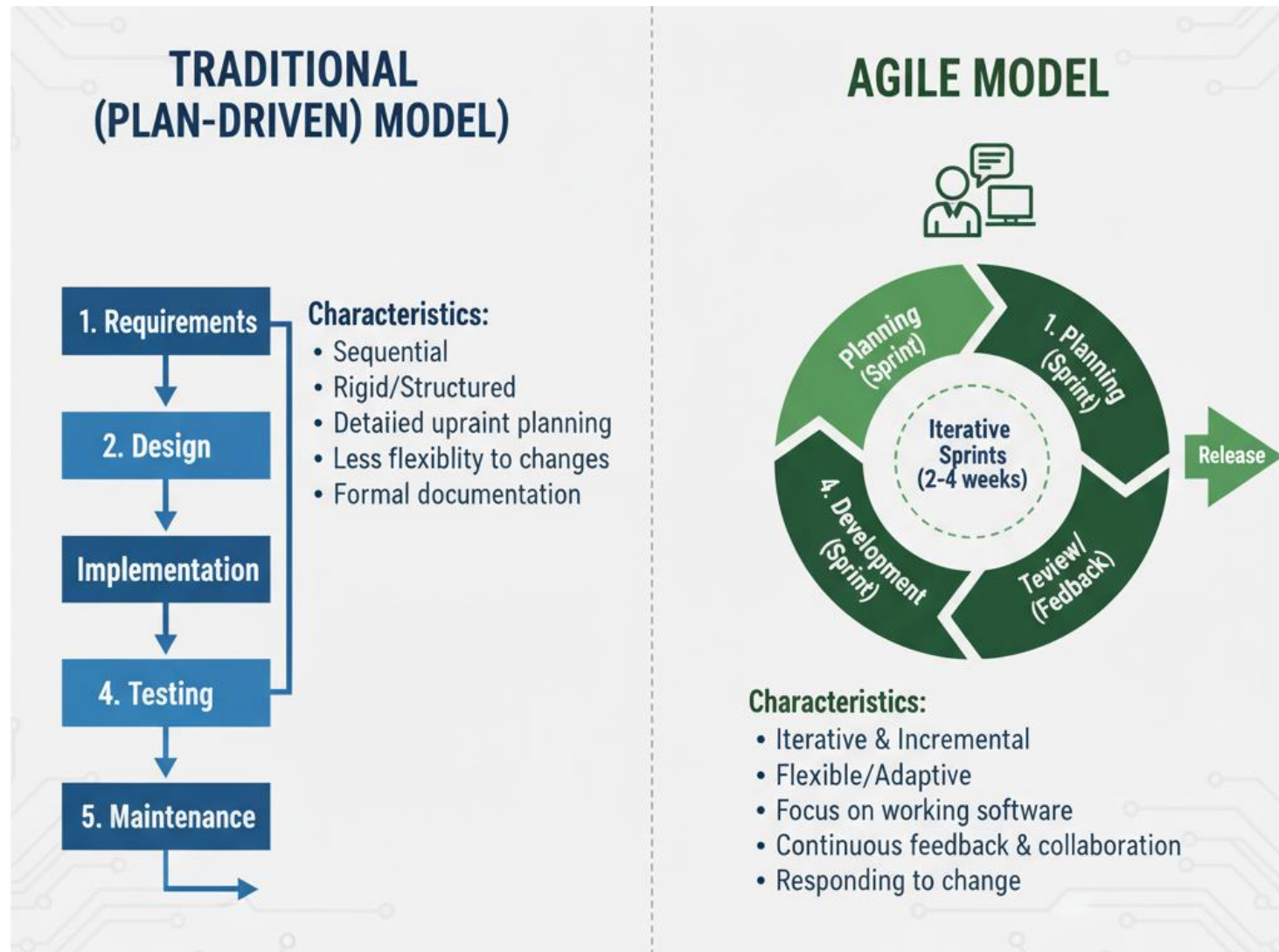
- Proposed by Boehm in 1988.
- Each loop of the spiral represents a phase of the software process:
 - the innermost loop might be concerned with system feasibility,
 - the next loop with system requirements definition,
 - the next one with system design, and so on.
- There are no fixed phases in this model, the phases shown in the figure are just examples.



Comparison of Plan-driven Process models

- Iterative waterfall model
 - most widely used model.
 - But, suitable only for well-understood problems.
- Prototype model is suitable for projects not well understood:
 - user requirements
 - technical aspects
- Evolutionary model is suitable for large problems:
 - can be decomposed into a set of modules that can be incrementally implemented,
 - incremental delivery of the system is acceptable to the customer.
- The spiral model:
 - suitable for development of technically challenging software products that are subject to several kinds of risks.

Plan-driven process models Vs Agile process models



Thank you !

What Is Agile Software Development?

- In the late 1990's several methodologies began to get increasing public attention. All emphasized:
 - Close collaboration between developers and business experts
 - Face-to-face communication (as more efficient than written documentation)
 - Frequent delivery of new deployable business value
 - Tight, self-organizing teams
 - Ways to craft the code and the team such that the inevitable requirements churn was not a crisis.
- 2001 : Workshop in Snowbird, Utah, Practitioners of these methodologies met to figure out just what it was they had in common. They picked the word "agile" for an umbrella term and crafted the [Manifesto for Agile Software Development](#),

Applying Agility

Effective (rapid and adaptive) response to change

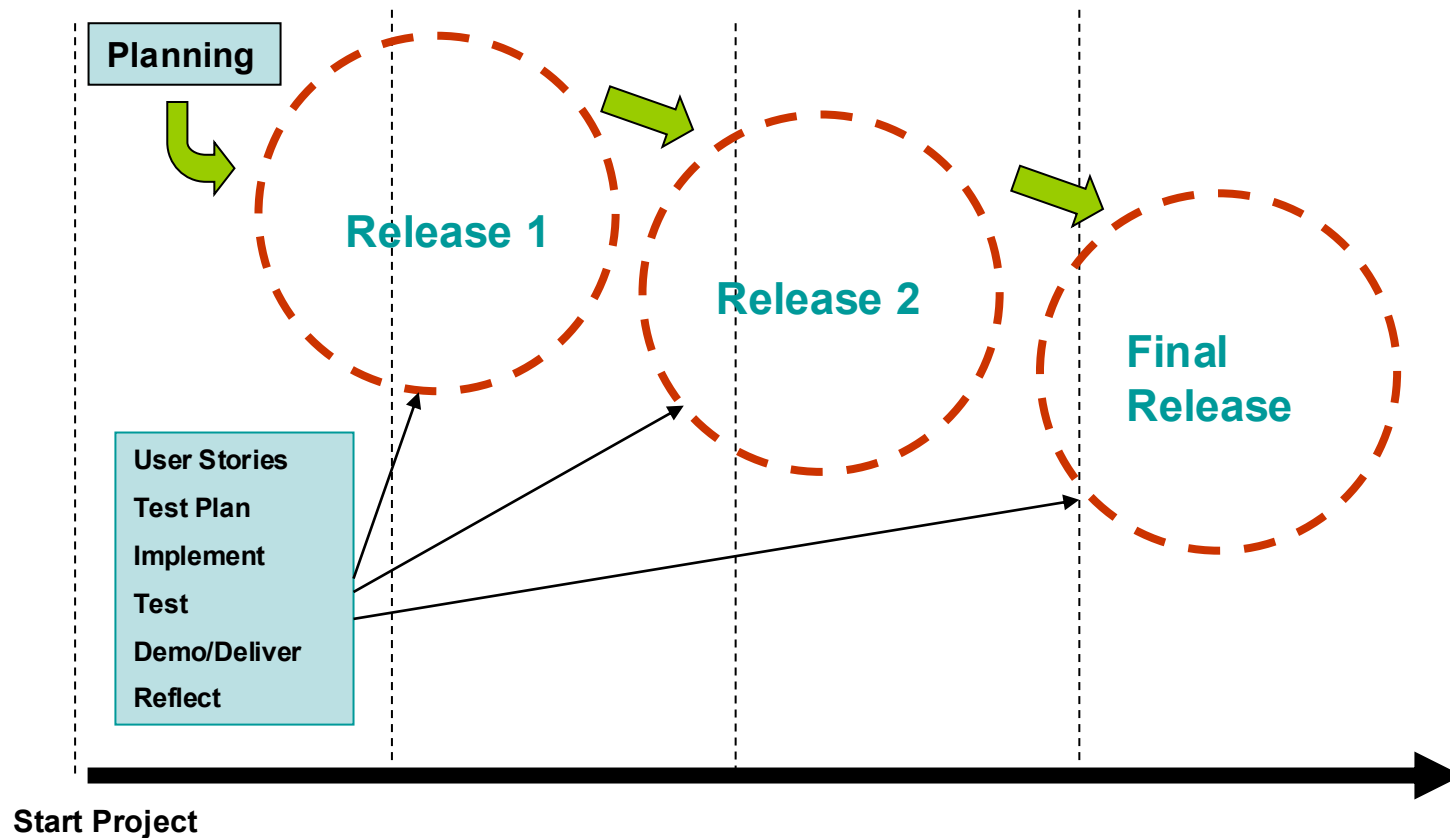
Effective communication among all stakeholders

Drawing the customer onto the team

Organizing a team so that it is in control of the work performed

Yielding ...

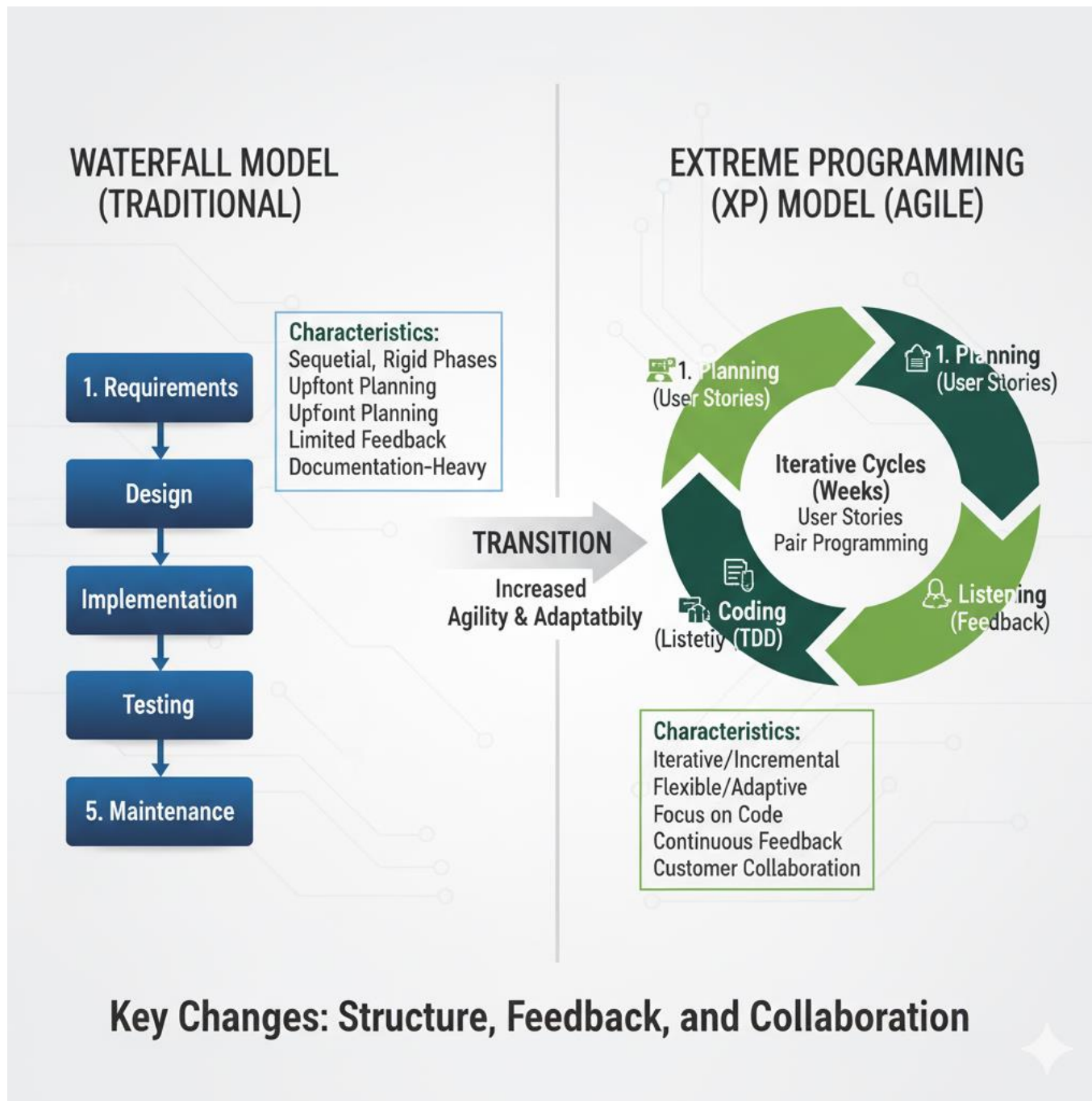
Rapid, incremental delivery of software



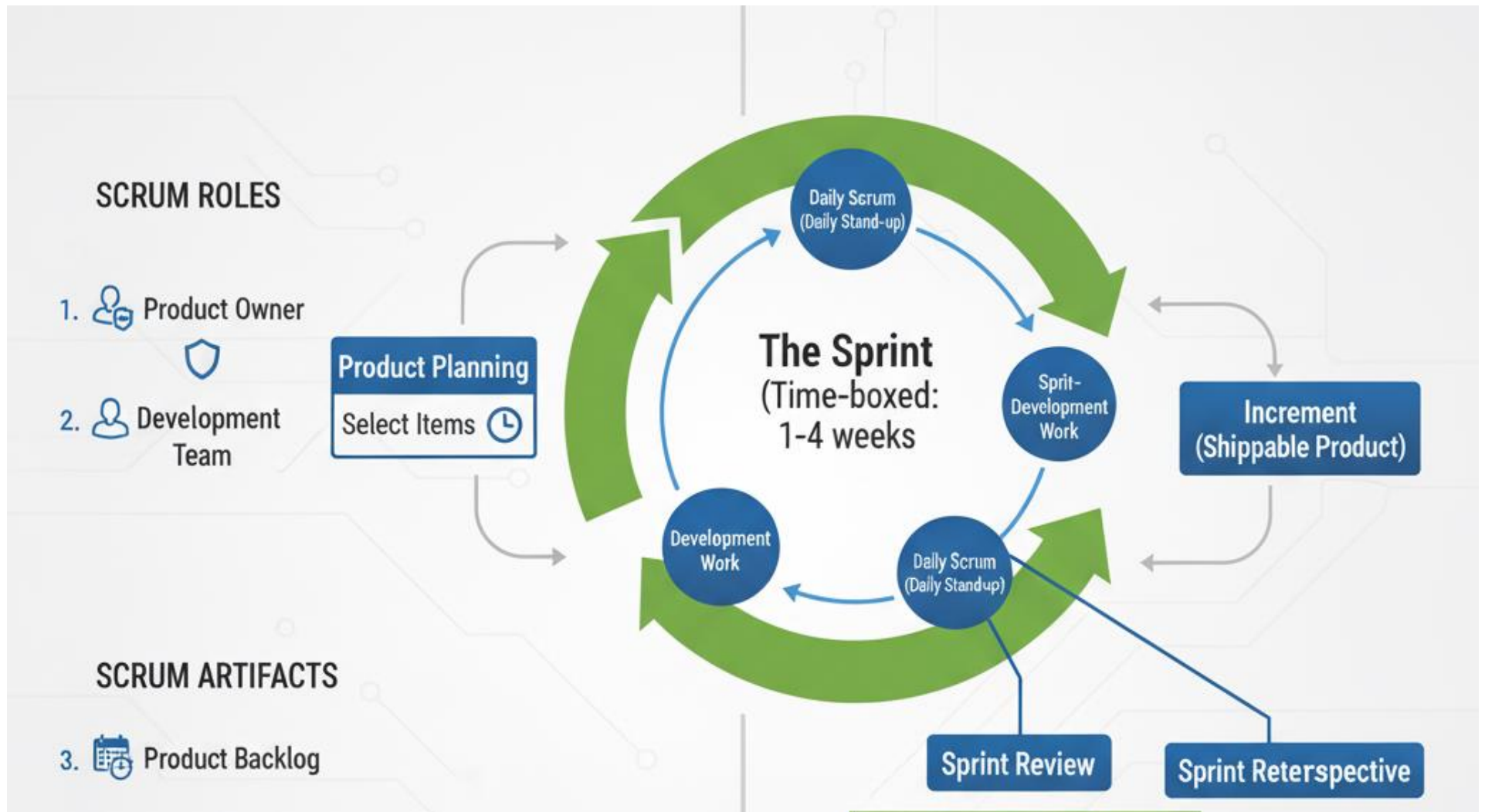
Agile Characteristics

- Incremental development – several releases
- Planning based on user stories
- Each iteration touches all life-cycle activities
- Testing – unit testing for deliverables; acceptance tests for each release
- Flexible Design – evolution vs. big upfront effort
- Reflection after each release cycle
- Several technical and customer focused presentation opportunities

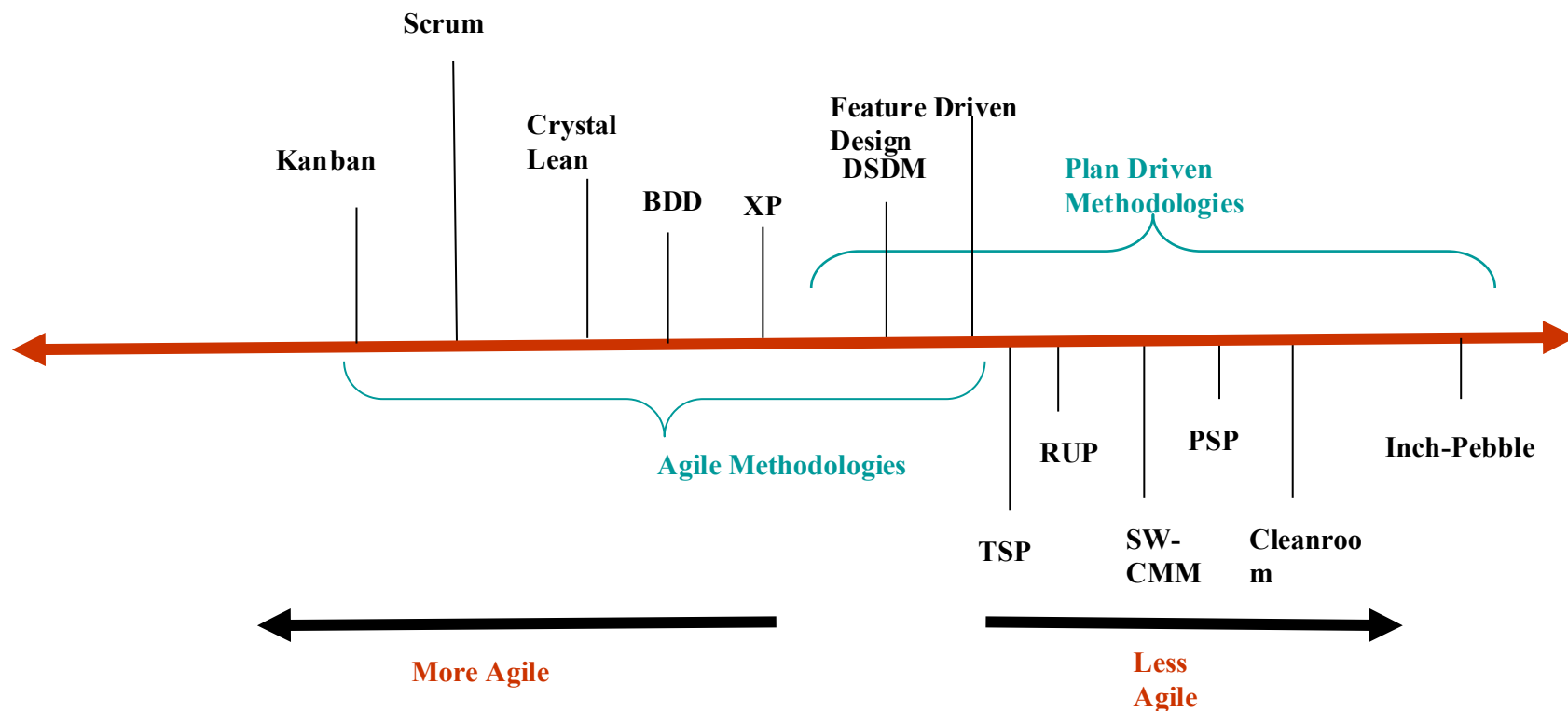
Waterfall to XP Evolution



Scrum Process



The Process Methodology Spectrum



It's not that black and white. The process spectrum spans a range of grey !

Thank you !

Key Agile Components

- User Stories
 - Requirements elicitation
 - Planning – scope & composition
- Evolutionary Design
 - Opportunity to make mistakes
- **Test driven development**
 - Dispels notion of testing as an end of cycle activity
- **Refactoring**
 - Small changes to code base to maintain design entropy
- **Continuous Integration**
 - Code (small booms vs big bang)
- Team Skills
 - Collaborative Development (Pair programming)
 - Reflections (process improvement)
- Communication/shared ownership
 - Interacting with customer / team members

TDD - example

```
import org.junit.Test;
import static org.junit.Assert.assertEquals;
```

```
public class CalculatorTest {

    @Test
    public void testAddition() {
        Calculator calculator = new
Calculator();
        int result = calculator.add(3, 4);
        assertEquals(7, result);
    }
}
```

```
public class Calculator {
```

```
    public int add(int a, int b) {
        return a + b;
    }
}
```

Write Code;

Test case

run



Test case

run



subtraction... (repeat the process)

@Test

```
public void testSubtraction() {
```

```
    Calculator calculator = new Calculator();
```

```
    int result = calculator.subtract (7, 4);
```

```
    assertEquals(3, result);
```

```
}
```

```
public class Calculator {
```

```
    public int add (int a, int b) {
```

```
        return a + b;
```

```
    }
```

```
    public int subtract (int a, int b) {
```

```
        return a - b;
```

```
    }
```

```
}
```

Implement the
functionality...

If you would like to add more functionality... but have common interface

```
import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class CalculatorTest {

    @Test
    public void testAddition() {
        Calculator calculator = new Calculator(new Addition());
        int result = calculator.calculate (3, 4);
        assertEquals(7, result);
    }

    @Test
    public void testSubtraction() {
        Calculator calculator = new Calculator(new Subtraction());
        int result = calculator.calculate (7, 4);
        assertEquals(3, result);
    }
}
```


REFACTOR

the code

```
public interface Operation {  
    int apply (int a, int b);  
}
```

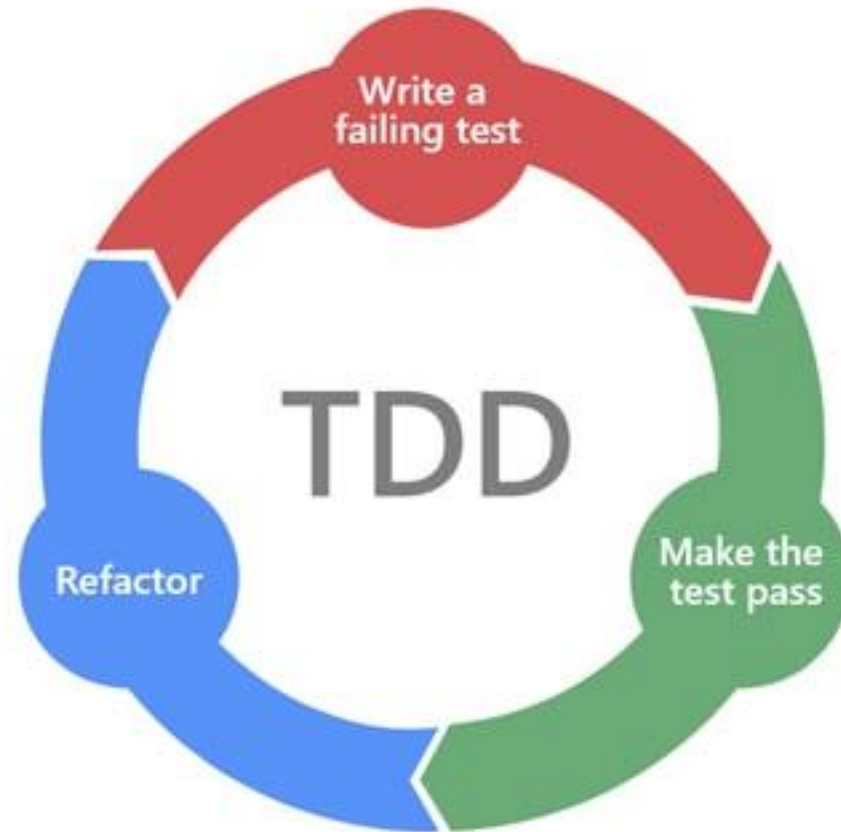
```
public class Addition implements Operation {  
    public int apply (int a, int b) {  
        return a + b;  
    }  
}
```

```
public class Subtraction implements Operation {  
    public int apply (int a, int b) {  
        return a - b;  
    }  
}
```

```
public class Calculator {  
  
    private final Operation operation;  
  
    public Calculator(Operation operation) {  
        this.operation = operation;  
    }  
  
    public int calculate(int a, int b) {  
        return operation.apply (a, b);  
    }  
}
```

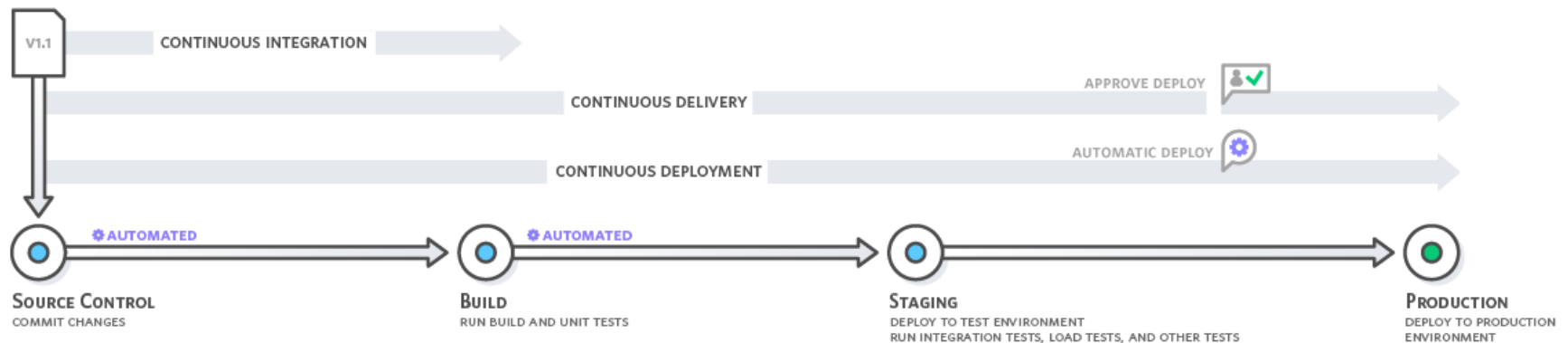
TDD Explained

RED –
GREEN –
REFACTOR
cycle



Continuous Integration/Continuous Deployment

- Developers merge their code into the central repository regularly
- Deploys all code changes to a testing and/or production environment after the build stage
- Automated builds and testing
- Deployment is Manual (Continuous Delivery) or automated (Continuous Deployment)



Developers & Operations teams need to work together...

Developers

- Designing
- Coding
- Testing, bug tracking, reviews
- Continuous Integration
- ...

Operations



Managing/Allocating
hardware/OS
updates/resources,
database



Monitoring
load spikes,
performance,
crashes
hardware
updates



Backups,
Rollback
releases



etc.

- ✓ Can there be better coordination between Developers and Operators?
- ✓ Reduce issues while moving changes from development to production
- ✓ Configurations as code
- ✓ Automation (Delivery and Monitoring)

DevOps – Common practices

- Continuous Integration
- Continuous Delivery
- Infrastructure as code, test and deploy in containers
- Monitoring and logging
- Microservice architecture
- Communicate and Collaborate

Thank you !